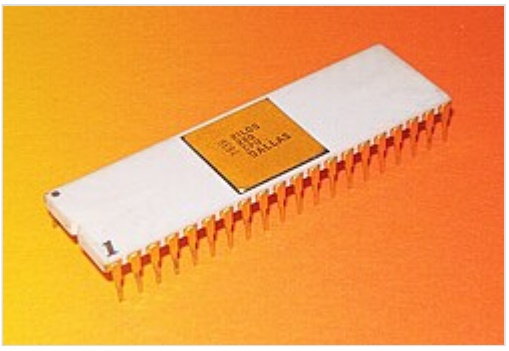
Z80 instruction set

The Zilog Z80 is an 8-bit microprocessor introduced in 1976. The instruction set was designed to be upward binary compatible with the Intel 8080. Intel 8080 instructions are one to three bytes long whereas the Z80 requires up to four bytes per instruction.



Zilog Z80 CPU. 1976 date code.

Zilog continued to expand the instruction set of the Z80 with several successors including the Z180, Z280, and Z380. The latest iteration, the eZ80, was introduced in 2001 and was available for purchase as of 2025. The instruction set also appears on non-Zilog CPUs such as the Hitachi HD64180,[1] Mitsui R800, and the Eastern Bloc U880.[2]

Instruction set

The Z80 uses 252 out of the available 256 codes as single byte opcodes (the "root instructions"), most of which are inherited from the 8080. The four remaining codes are used extensively as opcode prefixes:[3] CB and ED enable extra instructions, and DD or FD select IX or IY respectively in place of HL. This scheme gives the Z80 a large number of permutations of instructions and registers. Zilog categorizes these into 158 different "instruction types", 78 of which are the same as those of the Intel 8080.[3] This allows operation of all 8080 programs on a Z80. The Zilog documentation[4] further groups instructions into categories. Most are from the 8080, others are entirely new like the block and bit instructions, and other 8080 instructions with more versatile addressing modes, like the 16-bit loads, I/O, rotates/shifts, and relative jumps. The categories are:

- Load and exchange
- Block transfer and search
- Arithmetic and logical
- Rotate and shift
- Bit manipulation (set, reset, test)
- Jump, call, and return
- Input/output
- CPU control

Encoding order

To expand on the 8080 instruction set, Z80 instructions may require a IX/IY override, an opcode prefix, or both. Any one instruction may contain up to four components. The components of the instruction are assembled in the following order:[5]

IX/IY override
CB or ED prefix
(IX/IY + n) offset if CB
Opcode
(IX/IY + n) offset if no CB
data or address

Root instructions

The root opcodes include all the 8080 opcodes. The Z80 adds eight new one-byte instructions, two opcode prefixes, and the IX and IY overrides.[6] Colored rows indicate new Z80 instructions.

Opcode								Operands		Mnemonic	Description
7	**6**	**5**	**4**	**3**	**2**	**1**	**0**	**b2**	**b3**		
0	0	0	0	0	0	0	0	—	—	NOP	No operation
0	0	RP		0	0	0	1	*datlo*	*dathi*	LD *rp,data*	RP ← *data*
0	0	RP		0	0	1	0	—	—	LD *(rp)*,A	(RP) ← A [BC or DE only]
0	0	RP		0	0	1	1	—	—	INC *rp*	RP ← RP + 1
0	0	DDD			1	0	0	—	—	INC *ddd*	DDD ← DDD + 1
0	0	DDD			1	0	1	—	—	DEC *ddd*	DDD ← DDD - 1
0	0	DDD			1	1	0	*data*	—	LD *ddd,data*	DDD ← data
0	0	0	0	0	1	1	1	—	—	RLCA	$A_{1-7} \leftarrow A_{0-6}$; $A_0 \leftarrow Cy \leftarrow A_7$
0	0	RP		1	0	0	1	—	—	ADD *rp*	HL ← HL + RP
0	0	RP		1	0	1	0	—	—	LD A,*(rp)*	A ← (RP) [BC or DE only]
0	0	RP		1	0	1	1	—	—	DEC *rp*	RP ← RP - 1
0	0	0	0	1	0	0	0	—	—	EX AF,AF'	AF ↔ AF′
0	0	0	0	1	1	1	1	—	—	RRCA	$A_{0-6} \leftarrow A_{1-7}$; $A_7 \leftarrow Cy \leftarrow A_0$
0	0	0	1	0	0	0	0	offset	—	DJNZ *offset*	B = B - 1; if B ≠ 0 then PC ← PC + offset
0	0	0	1	0	1	1	1	—	—	RLA	$A_{1-7} \leftarrow A_{0-6}$; $Cy \leftarrow A_7$; $A_0 \leftarrow Cy$
0	0	0	1	1	0	0	0	offset	—	JR *offset*	PC ← PC + offset
0	0	0	1	1	1	1	1	—	—	RRA	$A_{0-6} \leftarrow A_{1-7}$; $Cy \leftarrow A_0$; $A_7 \leftarrow Cy$
0	0	1	CC		0	0	0	offset	—	JR *cc,offset*	If CC_{0-1} true, PC ← PC + offset (Only 2 bits of CC used: NZ, Z, NC, C)
0	0	1	0	0	0	1	0	*addlo*	*addhi*	LD *add,HL*	(add) ← HL
0	0	1	0	0	1	1	1	—	—	DAA	In N flag = 0 {If $A_{0-3} > 9$ OR AC = 1 then A ← A + 6; then if $A_{4-7} > 9$ OR Cy = 1 then A ← A + 0x60} else {do post-subtract DAA}
0	0	1	0	1	0	1	0	*addlo*	*addhi*	LD HL,*add*	HL ← (add)
0	0	1	0	1	1	1	1	—	—	CPL	A ← ¬A
0	0	1	1	0	0	1	0	*addlo*	*addhi*	LD *add*,A	(add) ← A
0	0	1	1	0	1	1	1	—	—	SCF	Cy ← 1
0	0	1	1	1	0	1	0	*addlo*	*addhi*	LD A,*add*	A ← (add)
0	0	1	1	1	1	1	1	—	—	CCF	Cy ← ¬Cy
0	1	DDD			SSS			—	—	LD *ddd,sss*	DDD ← SSS
0	1	1	1	0	1	1	0	—	—	HALT	Halt
1	0	ALU			SSS			—	—	ADD ADC SUB SBC AND XOR OR CP *sss*	A ← A [ALU operation] SSS

1	1	CC			0	0	0	—	—	RET *cc*	If cc true, PC ← (SP), SP ← SP + 2
1	1	RP		0	0	0	1	—	—	POP *rp*	RP ← (SP), SP ← SP + 2
1	1	CC			0	1	0	*addlo*	*addhi*	JP *cc,add*	If cc true, PC ← add
1	1	0	0	0	0	1	1	*addlo*	*addhi*	JP *add*	PC ← add
1	1	CC			1	0	0	*addlo*	*addhi*	CALL *cc,add*	If cc true, SP ← SP - 2, (SP) ← PC, PC ← add
1	1	RP		0	1	0	1	—	—	PUSH *rp*	SP ← SP - 2, (SP) ← RP
1	1	ALU			1	1	0	*data*	—	ADD ADC SUB SBC AND XOR OR CP *data*	A ← A [ALU operation] data
1	1	N			1	1	1	—	—	RST *n*	SP ← SP - 2, (SP) ← PC, PC ← N
1	1	0	0	1	0	0	1	—	—	RET	PC ← (SP), SP ← SP + 2
1	1	0	0	1	0	1	1	—	—	CB prefix	See CB prefix table
1	1	0	0	1	1	0	1	*addlo*	*addhi*	CALL *add*	SP ← SP - 2, (SP) ← PC, PC ← add
1	1	0	1	0	0	1	1	*port*	—	OUT *port*,A	Port(A:port) ← A[a]
1	1	0	1	1	0	0	1	—	—	EXX	BC ↔ BC′, DE ↔ DE′, HL ↔ HL′
1	1	0	1	1	0	1	1	*port*	—	IN A,*port*	A ← Port(A:port) [a]
1	1	0	1	1	1	0	1	—	—	IX override	HL becomes IX; (HL) becomes (IX + offset)
1	1	1	0	0	0	1	1	—	—	EX (SP),HL	(SP) ↔ HL
1	1	1	0	1	0	0	1	—	—	JP (HL)	PC ← HL
1	1	1	0	1	0	1	1	—	—	EX DE,HL	HL ↔ DE
1	1	1	0	1	1	0	1	—	—	ED prefix	See ED prefix table
1	1	1	1	0	0	1	1	—	—	DI	IFF1 ← IFF2 ← 0; Disable interrupts
1	1	1	1	1	0	0	1	—	—	LD SP,HL	SP ← HL
1	1	1	1	1	0	1	1	—	—	EI	IFF1 ← IFF2 ← 1; Enable interrupts
1	1	1	1	1	1	0	1	—	—	IY override	HL becomes IY; (HL) becomes (IY + offset)
7	**6**	**5**	**4**	**3**	**2**	**1**	**0**	**b2**	**b3**	**Mnemonic**	**Description**

SSS DDD	**2**	**1**	**0**	**CC**	**ALU**	**RP**
B	0	0	0	NZ	ADD (A ← A + arg)	BC
C	0	0	1	Z	ADC (A ← A + arg + Cy)	DE
D	0	1	0	NC	SUB (A ← A - arg)	HL
E	0	1	1	C	SBC (A ← A - arg - Cy)	SP or AF
H	1	0	0	PO	AND (A ← A ∧ arg)	
L	1	0	1	PE	XOR (A ← A ⊻ arg)	

(HL)	1	1	0	P	OR (A ← A ∨ arg)	
A	1	1	1	N	CP (A - arg)	
SSS DDD	**2**	**1**	**0**	**CC**	**ALU**	

a. OUT *port*,A and IN A,*port* instructions generate a 16-bit port address with the 8-bit immediate *port* number forming the lower part of the address and the A register forming the upper part. Typically, devices will only decode the lower part. In contrast, the 8080 duplicates the immediate *port* number on both the upper and lower address bus.

Instructions prefixed with ED

The ED-prefixed opcodes are a catch-all of new Z80 instructions that could not be encoded in one byte. This group encompasses only 60 of 256 available opcodes.[6]

Opcode								Operands		Mnemonic	Description
7	**6**	**5**	**4**	**3**	**2**	**1**	**0**	**b2**	**b3**		
0	1	DDD			0	0	0	—	—	IN *ddd*,(C)	DDD ← port(BC) [Except (HL)] (Port number is 16 bits)[a]
0	1	SSS			0	0	1	—	—	OUT (C),*sss*	port(BC) ← SSS [Except (HL)] (Port number is 16 bits)
0	1	RP		0	0	1	0	—	—	SBC HL,*ss*	HL ← HL – ss – CY
0	1	RP		0	0	1	1	*addlo*	*addhi*	LD *(add), ss*	(add) ← RP
0	1	0	0	0	1	0	0	—	—	NEG	A ← 0 - A
0	1	0	0	0	1	0	1	—	—	RETN	PC ← (SP); SP ← SP + 2; IFF1 ← IFF2[b]
0	1	0	NN		1	1	0	—	—	IM *n*	Interrupt mode 0, 1, 2 (encoded 0, 2, 3)
0	1	0	0	0	1	1	1	—	—	LD I,A	interrupt control vector ← A
0	1	RP		1	0	1	0	—	—	ADC HL,*ss*	HL ← HL + ss + CY
0	1	RP		1	0	1	1	*addlo*	*addhi*	LD *dd, (add)*	RP ← (add)
0	1	0	0	1	1	0	1	—	—	RETI	PC ← (SP); SP ← SP + 2; IFF1 ← IFF2[b]
0	1	0	0	1	1	1	1	—	—	LD R,A	refresh ← A
0	1	0	1	0	1	1	1	—	—	LD A,I	A ← interrupt control vector [c]
0	1	0	1	1	1	1	1	—	—	LD A,R	A ← refresh [c]
0	1	1	0	0	1	1	1	—	—	RRD	$A_{0-3} \leftarrow (HL)_{0-3}$; $(HL)_{7-4} \leftarrow A_{0-3}$; $(HL)_{0-3} \leftarrow (HL)_{7-4}$
0	1	1	0	1	1	1	1	—	—	RLD	$A_{0-3} \leftarrow (HL)_{7-4}$; $(HL)_{0-3} \leftarrow A_{0-3}$; $(HL)_{7-4} \leftarrow (HL)_{0-3}$
1	0	1	R	D	0	0	0	—	—	LDI LDIR LDD LDDR	(DE) ← (HL); HL ← HL ± 1; DE ← DE ± 1; BC ← BC - 1 [d][e]
1	0	1	R	D	0	0	1	—	—	CPI CPIR CPD CPDR	A - (HL); HL ← HL ± 1; BC ← BC - 1 [d][e][f]
1	0	1	R	D	0	1	0	—	—	INI INIR IND INDR	(HL) ← port(BC); HL ← HL ± 1; B ← B – 1 [d][g]
1	0	1	R	D	0	1	1	—	—	OUTI OTIR OUTD OTDR	B ← B – 1; port(BC) ← (HL); HL ← HL ± 1 [d][g][7]
7	**6**	**5**	**4**	**3**	**2**	**1**	**0**	**b2**	**b3**	**Mnemonic**	**Description**

a. Input byte sets the flags unlike IN A,n

b. RETN and RETI are identical and restore IFF1. Z80 compatible interrupt devices watch for RETI by sniffing the data bus while M1- is asserted for 0xED followed by 0x4D.

c. LD A,I and LD A,R are the only two LD instructions that set flags. Additionally, IFF2 is loaded into the P/V flag. C unaffected.

d. When D = 1, pointers HL and DE decrement. When R = 1, operation repeats until BC or B = 0.

e. LDI, LDD, CPI, and CPD set P/V if BC – 1 ≠ 0. This is useful for loop control when not using

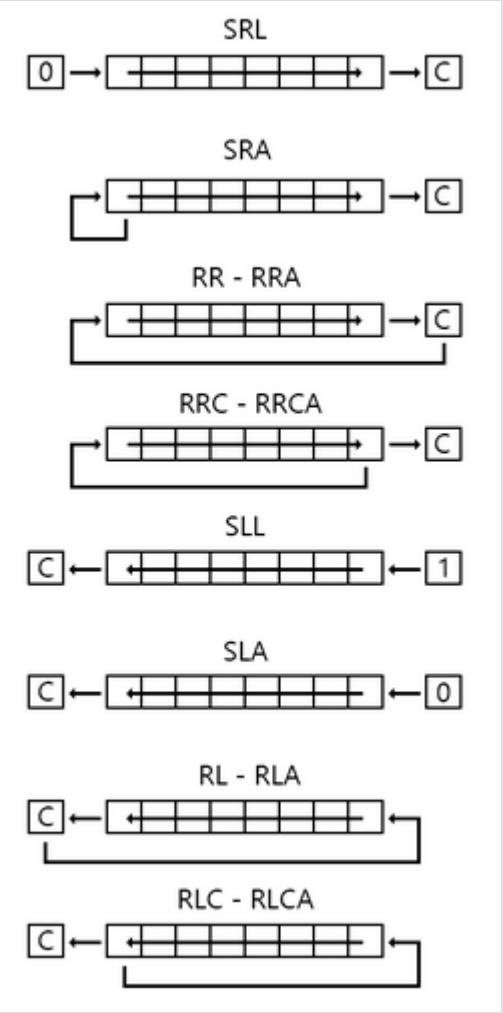
repeat.

f. CPIR/CPDR terminate early if A = (HL).
g. All block IO instructions output BC, not just C, as the port address.

Instructions prefixed with CB

The CB-prefixed opcodes cover shifts and rotates plus the bit test, clear, and set instructions. All of these instructions can be used with any register or memory. The (HL) form of these instructions can be combined with an IX or IY opcode prefix to operate on (IX+d) or (IY+d). This group encompasses 248 of 256 available opcodes.[6]

Opcode								Mnemonic	Description
7	**6**	**5**	**4**	**3**	**2**	**1**	**0**		
0	0	0	0	0	SSS			RLC *r*	$R_{1-7} \leftarrow R_{0-6}$; $R_0 \leftarrow Cy \leftarrow R_7$
0	0	0	0	1	SSS			RRC *r*	$R_{0-6} \leftarrow R_{1-7}$; $R_7 \leftarrow Cy \leftarrow R_0$
0	0	0	1	0	SSS			RL *r*	$R_{1-7} \leftarrow R_{0-6}$; $Cy \leftarrow R_7$; $R_0 \leftarrow Cy$
0	0	0	1	1	SSS			RR *r*	$R_{0-6} \leftarrow R_{1-7}$; $Cy \leftarrow R_0$; $R_7 \leftarrow Cy$
0	0	1	0	0	SSS			SLA *r*	$Cy \leftarrow R_7$; $R_{1-7} \leftarrow R_{0-6}$; $R_0 \leftarrow 0$
0	0	1	0	1	SSS			SRA *r*	$Cy \leftarrow R_0$; $R_{0-6} \leftarrow R_{1-7}$
0	0	1	1	1	SSS			SRL *r*	$Cy \leftarrow R_0$; $R_{0-6} \leftarrow R_{1-7}$; $R_7 \leftarrow 0$
0	1	bit			SSS			BIT *b,r*	R ∧ (1 << b)
1	0	bit			SSS			RES *b,r*	R ← R ∧ ¬(1 << b)
1	1	bit			SSS			SET *b,r*	R ← R ∨ (1 << b)
7	**6**	**5**	**4**	**3**	**2**	**1**	**0**	**Mnemonic**	**Description**



Shift and rotate instructions. SLL is an undefined instruction.

IX and IY overrides

Two opcode prefixes expand the number of Z80 addressing modes to access the new IX and IY index registers:

- Prefix DD changes HL to IX or (HL) to (IX+displacement)
- Prefix FD changes HL to IY or (HL) to (IY+displacement)

The index registers, IX and IY, were intended as flexible 16-bit pointers, enhancing the ability to manipulate memory, stack frames, and data structures. Officially, they were treated as 16-bit only. In reality, they were implemented as a pair of 8-bit registers[8] in the same fashion as the HL register, which is accessible either as 16 bits or separately as the high and low registers. The binary opcodes were identical, but preceded by a new opcode prefix.[9]

Undocumented uses of IX/IY

Zilog published the opcodes and related mnemonics for the intended functions, but did not document the fact that every opcode that allowed manipulation of the HL register was equally valid for the 8-bit halves of the IX and IY registers. For example, opcode 26h followed by an immediate byte value forms the instruction `LD H,n`. It will load the *n* immediate value into the H register. Preceding this two-byte instruction with the IX register's opcode prefix, DD, would instead result in the most significant 8 bits of the IX register being loaded with that same value. A notable exception to this would be instructions similar to `LD H,(IX+d)` which make use of both the HL and IX or IY registers in the same instruction.[9] In this case the DD prefix is only applied to the (IX+d) portion of the instruction.

Accessing the IX/IY halves can speed some operations. To load DE into IX using official instructions, one could use `PUSH DE` and `POP IX`, taking 25 cycles. Using the half-load feature, the same could be coded as `LD IXL,E` and `LD IXH,D`, saving 9 cycles but taking an extra byte.

The IX and IY halves can also hold operands for 8-bit arithmetic, logical, and compare instructions, sparing the regular 8-bit registers for other use. Any IX/IY half can be incremented or decremented or loaded to or from any 8-bit register except H or L.[9]

Instruction synonyms

The Z80 expands the 8080 instruction set and makes it more orthogonal. A byproduct of this is there are certain instruction functions that map to two different instruction encodings. In a few cases there is an 8080 instruction coding that is smaller and faster than the orthogonal Z80 encoding.[5] Some examples:

- The Z80 introduced orthogonal versions of `LD RP,(nn)` and `LD (nn),RP` instructions where RP can be any register pair or SP. The Z80 assembler substitutes the 8080 version of `LD HL,(nn)` and `LD (nn),HL` to save a byte and four clocks.
- `RL A`, `RR A`, `RLC A`, `RRC A`, and `SLA A` instructions are part of the Z80 CB-prefixed rotates. The 8080 `RLA`, `RRA`, `RLCA`, `RRCA`, and `ADD A,A` instructions are a byte smaller and twice as fast.
- The Z80 introduced a two-byte unconditional jump, `JR`. Although larger, the three-byte 8080 `JP` is two clocks faster.
- The Z80 added two-byte conditional jumps: `JR NZ`, `JR Z`, `JR NC`, and `JR C` as alternates to the 8080 three byte conditional jumps: `JP NZ`, `JP Z`, `JP NC`, and `JP C`. Although the two-byte forms are two clocks slower when taken, they are three clocks faster when not taken.
- The Z80 DJNZ instruction is half the size of the 8080 equivalent instructions and one clock faster.

References

1. Cohen, Charles L. (4 March 1985). "Sun Rises on New Designs" (https://archive.org/details/electronics_week-1985_03_04/page/18/mode/2up). *ElectronicsWeek*. pp. 18–21. Retrieved 2 June 2024.
2. "UD 880 und UD 855" [UD 880 and UD 855]. *Radio Fernsehen Elektronik* (in German). **35** (2). VEB Verlag Technik: 70. 1986. ISSN 0033-7900 (https://search.worldcat.org/issn/0033-7900).
3. "Z80 CPU Introduction" (http://www.z80.info/z80brief.htm). Zilog. 1995. Archived (https://web.archive.org/web/20231220144519/http://www.z80.info/z80brief.htm) from the original on December 20, 2023. "It has a language of 252 root instructions and with the reserved 4 bytes as prefixes, accesses an additional 308 instructions."
4. "Z80-CPU Instruction Set" (https://usermanual.wiki/Document/ZilogZ80CPUTechnicalManual.2406416115/view#23) (PDF). Zilog. 1976. p. 19. Archived (https://web.archive.org/web/20231105184525/https://usermanual.wiki/Document/ZilogZ80CPUTechnicalManual.2406416115/view#23) from the original on November 5, 2023. Retrieved July 20, 2021.
5. "Z80" (https://www.cpcwiki.eu/index.php/Z80). *CPC Wiki*. Retrieved 6 September 2025.
6. *Z80 CPU User Manual* (https://www.zilog.com/docs/z80/um0080.pdf) (PDF) (11 ed.). Zilog. August 2016. Retrieved 15 August 2025.
7. "Z80 Documentation Errors" (https://www.cpcwiki.eu/forum/programming/z80-documentation-errors/). *CPC Wiki*. Retrieved 28 November 2025. Unlike stated in Z80 documentation, OUTI OTIR OUTD OTDR decrement B before the IO access.
8. Froehlich, Robert A. (1984). *The free software catalog and directory* (https://archive.org/details/freesoftwarecata0000froe/). Crown Publishers. p. 133. ISBN 978-0-517-55448-7. "Undocumented Z80 codes allow 8-bit operations with IX and IY registers."
9. Bot, Jacco J. T. "Z80 Undocumented Instructions" (http://www.z80.info/z80undoc.htm). *Home of the Z80 CPU*. Archived (https://web.archive.org/web/20231223185034/http://z80.info/z80undoc.htm) from the original on December 23, 2023. "If an opcode works with the registers HL, H or L then if that opcode is preceded by #DD (or #FD) it works on IX, IXH or IXL (or IY, IYH, IYL), with some exceptions. The exceptions are instructions like LD H,IXH and LD L,IYH."